

Debugging

Lỗi thường gặp:

- + tên biến sai hoặc chưa gán giá trị.
- + sai thứ tự phép toán
- + quên dấu ngoặc hoặc dấu nháy

Modulus & String

% trả phần dư
Chuỗi: + nối, * lặp

Value & Type

Dữ liệu cơ bản: số, chuỗi, ký tự
Kiểu dữ liệu: int, float, str
Kiểm tra kiểu: type(x)

Expression & Order

Biểu thức = tổ hợp toán tử, biến, giá trị
Quy tắc ưu tiên: PEMDAS

Operators & Operands

Toán tử: +, -, *, /,
%(chia lấy dư), //(chia lấy nguyên),
**(lũy thừa)
Operand = giá trị bị tác động

Statement

Lệnh python được thực thi (VD:
print(x))

Variables, Expressions and Statements

Variable

Đại diện cho giá trị
Gán giá trị =

Input & Comments

Nhập dữ liệu từ bàn
phím: input()
Ghi chú: #

Tên biến & từ khóa

Tên biến không bắt đầu bằng số,
không chứa ký tự đặc biệt
Có 33 keywords (and, if, for,...)

Câu lệnh if

Cấu trúc cơ bản:

```
✓ if x > 0:  
    print("Positive")
```

Thực thi khi điều kiện đúng

Biểu thức điều kiện

Kết quả là True hoặc False

Toán tử so sánh:

==, !=, >, <, >=, <=

Câu lệnh try/except

Dùng để bắt lỗi nhập sai hoặc xử lý ngoại lệ:

```
try:  
    value = int(input("Enter number: "))  
except:  
    print("Invalid input")
```

Gỡ lỗi

Kiểm tra kỹ điều kiện (== vs =)

Thêm print() để theo dõi luồng điều khiển

Sử dụng từng bước nhỏ để xác minh logic

Conditional Execution

Câu lệnh if-else

Dùng khi có hai khả năng:

```
if x % 2 == 0:  
    print("Even")  
else:  
    print("Odd")
```

Toán tử logic

Dùng để kết hợp nhiều điều kiện:

and → cả hai đúng

or → một trong hai đúng

not → phủ định

Câu lệnh chồng lên nhau

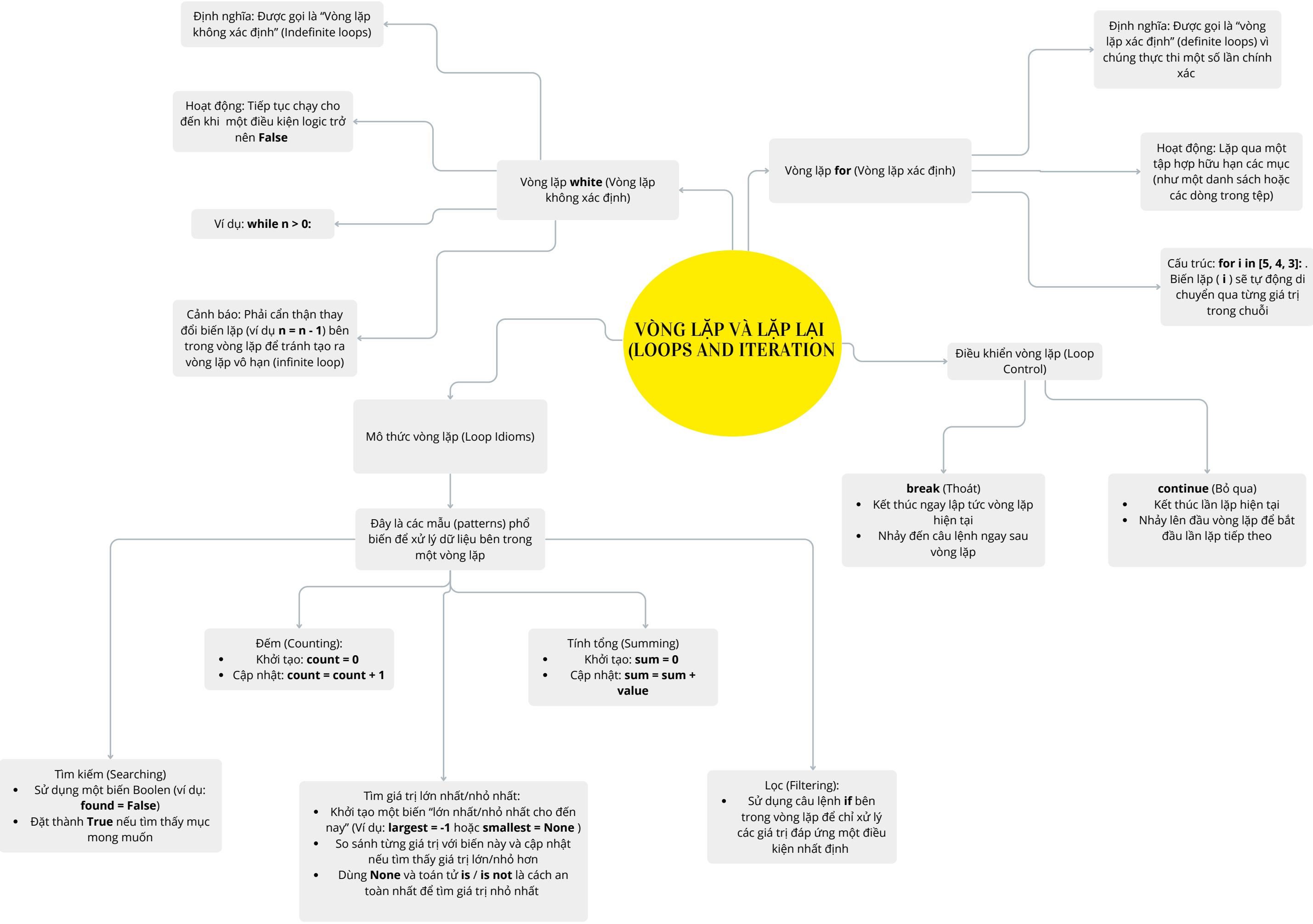
Đặt if bên trong if khác để đọc hơn nếu chuyển sang dùng elif khi có thể

Câu lệnh if-elif-else (Chuỗi điều kiện)

Dùng cho nhiều trường hợp:

```
if x < 0:  
    print("Negative")  
elif x == 0:  
    print("Zero")  
else:  
    print("Positive")
```





HÀM

-Hàm(function) là khối mã có thể lưu trữ & tái sử dụng, định nghĩa bằng def.

Gọi hàm

```
def say_hi():  
    print("HELLO!")  
  
say_hi()
```

Tham số & đối số:
+positional: theo đúng thứ tự
+keyword: gọi theo tên
+default: có giá trị mặc định
+multiple: tham số tùy ý

Biến:
+parameter=biến định nghĩa trong hàm
+argument=giá trị truyền vào khi gọi hàm.

Cấu trúc: store&reuse:
-viết một lần, gọi nhiều lần, giúp code ngắn gọn hơn.

Phạm vi biến:
Local: khai báo trong hàm
Global: khai báo ngoài hàm
Nonlocal: dùng trong hàm lồng nhau

hàm lồng nhau:

```
def say():  
    print("Đây là hàm 1")  
    def answer():  
        print("Đây là hàm 2 (nằm trong hàm 1)")  
    answer()  
say()
```

Hàm có sẵn
-print(), input(), type(), int(), float().....

Return
-return kết thúc hàm và trả về giá trị.
-không có return trả về none.

Type conversion:
-int(), float(), str()=> chuyển đổi kiểu dữ liệu
-lỗi nếu chuỗi không phải số (Value Error)

STRING

1. Khái niệm	2. Độ dài chuỗi	3. Duyệt chuỗi
Chuỗi là dãy kí tự Truy cập từng kí tự bằng chỉ số (index) bắt đầu từ số 0 <pre>fruit = "banana" print(fruit[0]) print(fruit[1])</pre>	Hàm len() trả về số kí tự trong chuỗi Kí tự cuối có index=len(string)-1 <pre>fruit = "banana" print(len(fruit)) # 6 print(fruit[-1]) # a</pre>	Dùng while hoặc for để duyệt từng kí tự <pre>for c in "banana": print(c)</pre>
4. Cắt chuỗi	5. Toán tử và so sánh chuỗi	6. Các phương thức xử lý
Cú pháp string[start:end] (lấy từ start đến end-1) <pre>s = "Monty Python" print(s[0:5]) # Monty print(s[6:]) # Python</pre>	In, not in kiểm tra chuỗi con <pre>"py" in "python" # True</pre> So sánh theo bảng chữ cái, phân biệt hoa – thường.	<pre>s.upper()</pre> : Viết hoa toàn bộ <pre>s.lower()</pre> : Viết thường <pre>s.find('a')</pre> : Vị trí đầu tiên <pre>s.count('a')</pre> : Số lần xuất hiện <pre>s.strip()</pre> : Xóa khoảng trắng 2 đầu <pre>s.replace('a','o')</pre> : Thay thế ký tự <pre>s.split()</pre> : Tách chuỗi thành list
	7. Xử lý ký tự, kiểm tra	
	<pre>s.isalpha()</pre> : Toàn chữ cái? <pre>s.isdigit()</pre> : Toàn số? <pre>s.islower()</pre> : Chữ thường? <pre>s.isupper()</pre> : Chữ hoa?	

FILES

1. Persistence	2. Opening Files	3. Text Files and Lines
Dữ liệu trong CPU và RAM mất khi tắt máy, còn file trong bộ nhớ phụ (ổ đĩa, USB) thì không. File giúp lưu trữ và tái sử dụng dữ liệu giữa các lần chạy chương trình.	<pre>fhand = open('mbox.txt')</pre> Open tạo file handle— một “tay cầm” để làm việc với dữ liệu. Nếu file không tồn tại → python báo lỗi FileNotFoundError	File văn bản = chuỗi các dòng, mỗi dòng kết thúc bằng kí tự xuống dòng \n \n là 1 kí tự duy nhất biểu diễn kết thúc dòng khi đọc file
4. Reading Files	5. Searching in Files	6. User Input File Name
<pre>for line in fhand: count += 1</pre> Có thể duyệt từng dòng bằng vòng lặp for Read() đọc toàn bộ nội dung file thành một chuỗi	Duyệt từng dòng và lọc điều kiện bằng startswith() hoặc find <pre>for line in fhand: line = line.rstrip() if line.startswith('From:'): print(line)</pre>	<pre>fname=input('Enter file name:') fhand = open(fname)</pre> Cho phép người dùng tự chọn file để xử lý. Dùng lặp lại chương trình cho các file khác nhau mà không cần sửa code.
7. Try/Except khi mở file		8. Writing Files
<pre>try: fhand = open(fname) except: print('File cannot be opened:', fname) exit()</pre> Tránh lỗi khi file không tồn tại. try/except giúp chương trình chạy mượt, không bị crash		<pre>fout = open('output.txt', 'w') fout.write("Hello\n") fout.close()</pre> Dùng chế độ “w” để ghi file Nếu file đã có → bị xóa dữ liệu cũ. Dùng close() để đảm bảo dữ liệu được lưu hoàn toàn.

-List là một cấu trúc dữ liệu cho phép lưu nhiều giá trị trong một biến bên trong [] và phân cách bởi dấu phẩy.
-List có thể thay đổi và có thể rỗng.

Các phương thức xóa của list:
-pop(i)=>xóa và trả về phần tử i (nếu không có i pop sẽ tự động trả về phần tử cuối)
-remove(x)=>xóa phần tử đầu tiên có giá trị x
-del my_list[i]=>xóa phần tử tại vị trí i
-clear=>xóa toàn bộ list..

Tương tác với chuỗi:
-split()=>tách string thành list(mặc định tách theo khoảng trắng)
- join() => nối từ list lại thành string. dùng sep làm dấu nối

-Các phương thức list:
+append(x)=>thêm phần tử vào cuối danh sách
+insert(i,x)=>chèn phần tử vào một vị trí i.
+sort()=>sắp xếp tăng dần
+count(x)=>đếm số lần xuất hiện
+reverse()=>đảo ngược thứ tự
+index(x)=> trả về vị trí đầu tiên của x

Các hàm tích hợp:
Len(list)=>đếm số phần tử
'+'=>nối 2 list
list * n=>lặp lại list n lần
in và not in=>kiểm tra phần tử=>true hay false.
Sum(list)=> tính tổng các phần tử
Max(list) và min(list)=>phần tử lớn nhất/nhỏ nhất

Truy cập(indexing) và cắt lát(slicing):
-.bắt đầu từ số 0 và số âm dùng để đếm ngược=>dùng để truy cập.
-cú pháp slicing [start:stop:step]

```
my_list=['a','b','c','d']  
print(my_list[0]) # a  
print(my_list[-1]) # d
```

```
my_list=[0,1,2,3,4,5]  
print(my_list[::2]) # => 0,2,4
```

LIST

```
test='one two there'  
abc=test.split()  
print(abc)
```

['one', 'two', 'there']

Duyệt list:
Dùng hàm for:

```
test=['one','two','there']  
for i in test:  
    print(i)
```

one
two
there

Numpy:

-Xử lý mảng và các phép tính số học

- np.array=>tạo mảng
- np.zeros/np.ones=>tạo ma trận toàn 0/1
- np.mean=>tính trung bình
- np.std=>độ lệch chuẩn
- np.dot(a,b)=>nhân ma trận
- np.reshape=>đổi kích thước ma trận
- np.arange(strat,stop,step)=>tạo dãy số cách đều nhau một khoảng
- np.linspace(strat,stop,num)=>chia đều một khoảng thành N phần
- arr.shape=>xem kích thước của mảng

Scikit-learn:

-dùng để huấn luyện và đánh giá mô hình

- model= Linear Regression=>tạo mô hình
- model.fit(X_train,y_train)=>huấn luyện
- y_pred=model.predict(X_test)=>dự đoán
- model.score(X_test,y_test)=>đánh giá độ chính xác

Pytorch:

-Xây dựng và huấn luyện mạng nơ-ron,chạy GPU

- x=torch.randn=>tạo tensor ngẫu nhiên
- x.cuda=>chuyển tensor sang gpu
- x.backward=>lan truyền ngược.

Pandas:

-thao tác và phân tích dữ liệu có cấu trúc

- df.head=>xem 5 dòng đầu
- df.info=>thông tin cột và kiểu dữ liệu
- df.describe=>thống kê cơ bản
- df['col']=>truy cập một cột
- df.iloc[0]=>lấy một dòng theo chỉ số
- df.sort_values=>sắp xếp dữ liệu
- df.dropna=>xóa dòng bị thiếu dữ liệu
- df.fillna=>điền giá trị vào ô trống
- pr.read_csv('file.csv')=>đọc file csv
- df.to_csv('file_moi.csv')=>lưu data thành file csv
- df.groupby('tên cột')=>nhóm dữ liệu

Những thư viện quan trọng của python

Matplotlib:

-Vẽ biểu đồ trực quan hóa dữ liệu dễ hiểu

- plt.plot(x,y)=>vẽ đường
- plt.bar(x,y)=>vẽ cột
- plt.scatter(x,y)=>vẽ điểm
- plt.hist(x)=>vẽ biểu đồ tần suất
- plt.title=>tiêu đề biểu đồ
- plt.x/ylabel=> nhãn trục/x/y
- plt.show=>hiện thị biểu đồ
- plt.savefig('ten_bieu_do.png')=>lưu biểu đồ thành file ảnh
- plt.legend()=>hiện thị chú thích

Opencv:

-xử lý ảnh và video

- cv2.imshow=>hiện thị ảnh
- cv2.waitKey=>dừng hiện thị
- cv2.imwrite=>lưu ảnh
- img=cv2.imread=>đọc ảnh

NLTK:

Xử lý văn bản và phân tích câu từ

- from nltk.tokenize import word_tokenize=>tách từ
- from nltk.corpus import stopwords=>gọi danh sách từ dừng
- stopwords.words('english')=>danh sách từ dừng tiếng anh